Project Assignment

AIT 203: Optimization

Question 1

Bike Sharing Demand Regression Analysis

Team Members:
BT2024212 Aryan Sharma
BT2024028 Sutaria Parth Anandkumar

1. Preprocessing and Prevention of Train/Test Leakage

The objective is to predict the hourly **bike rental count** using regression models implemented **from scratch** (normal equation).

To avoid leakage, the following steps were taken:

- The `datetime` column was decomposed into hour, day, month, and year, and then removed.
- The columns `casual` and `registered` were dropped because

  $$\texttt{count} = \texttt{casual} + \texttt{registered},$$

  meaning they directly reveal the target and would cause leakage.
- Categorical features (season, weather, workingday, etc.) were one-hot encoded to avoid imposing artificial numeric order.
- All features were standardized using the mean and variance computed **only from the training set**, ensuring no leakage.
- A random 80–20 split was applied **before** any preprocessing.

2. Regression Models Implemented

All regression coefficients were computed using the normal equation:

$$\beta = (X^T X)^\dagger X^T y.$$

The following models were evaluated:

1. **Linear Regression (baseline)**
2. **Polynomial Regression (degrees 2, 3, 4) without interaction terms**, i.e.,

   $$[x, x^2, x^3, \ldots, x^d]$$

   for each feature independently.

3. **Quadratic Polynomial Regression with Interaction Terms**:

$$x_i, \quad x_i^2, \quad x_i x_j \ \ (i < j),$$

capturing cross-feature effects.

3. Test-Set Performance Metrics

Mean Squared Error (MSE) and R^2 were computed as:

$$\text{MSE} = \frac{1}{n}\sum(y_i - \hat{y}_i)^2, \qquad R^2 = 1 - \frac{\sum(y_i - \hat{y}_i)^2}{\sum(y_i - \bar{y})^2}.$$

Model	**MSE**	$\mathbf{R^2}$
Linear Regression	9768.33	0.7012
Polynomial (Degree 2, No Interactions)	9696.04	0.7034
Polynomial (Degree 3, No Interactions)	9522.84	0.7087
Polynomial (Degree 4, No Interactions)	9489.17	0.7098
Quadratic w/ Interaction Terms	**2831.19**	**0.9134**

4. Best Model and Explanation

The **Quadratic Polynomial Regression with interaction terms** is the best-performing model, achieving:

$$\text{MSE} = 2831.19, \qquad R^2 = 0.9134.$$

This large improvement can be explained as follows:

- **Bike demand depends on combinations of features, not single features alone.** For example, rental demand is high at 8–9 AM on *working days*, but not on weekends. This behavior is captured by the interaction term:

$$\text{hour} \times \text{workingday}.$$

- **Weather effects also interact.** Temperature alone does not determine comfort; humidity modifies how hot it feels. The interaction term:

$$\text{temperature} \times \text{humidity}$$

  helps the model learn why demand drops when high heat combines with high humidity.

- **Season and weather interact.** Light rain in summer affects riders differently than the same rain in winter. This is captured by:

$$\text{season} \times \text{weather}.$$

- **Polynomial models without interactions only bend curves, but cannot model such relationships.** They only add terms like x^2, x^3, x^4, which slightly improve curvature but still treat each feature independently. This explains why their R^2 improves only from 0.7012 to about 0.7098.

- **Quadratic interaction terms introduce the minimum needed nonlinearity.** They allow the model to capture real-world effects while avoiding the high variance of deeper polynomial models.

Thus, the quadratic interaction model strikes the right balance between flexibility and stability, and best reflects the true structure of bike rental demand.

5. Conclusion

All models were implemented manually using NumPy, and evaluated on a clean, leakage-free test set. Polynomial models without interactions yielded only small improvements (R^2 up to 0.7098). In contrast, the quadratic model with interaction terms produced a significant performance gain, achieving $R^2 = 0.9134$.

This demonstrates that interaction effects play a crucial role in predicting bike rental demand, and incorporating them leads to superior model performance.

```python
import pandas as pd
import numpy as np

# LOAD TRAINING DATA
df = pd.read_csv("/kaggle/input/bike-sharing-demand/train.csv",
parse_dates=["datetime"])

# Parsing Date and Time
df["hour"] = df["datetime"].dt.hour
df["day"] = df["datetime"].dt.day
df["month"] = df["datetime"].dt.month
df["year"] = df["datetime"].dt.year
df = df.drop(columns=["datetime"])


# Remove columns that cause leakage because in real life we wont have
these fields as input, because if
# we do have them, that means we have the answer itself ,  count =
casual + registered
df = df.drop(columns=["casual", "registered"])

# print(df.head())

# WHY ONE HOT ENCODE??
# One-hot encoding prevents linear regression from assuming an
incorrect numeric ordering between categories.
# If we use weather = 1, 2, 3, 4 directly
# Is weather = 4 really "4 times worse/better" than weather = 1?

def one_hot_encode(df, col_name):
    unique_vals = sorted(df[col_name].unique())
    for val in unique_vals:
        df[f"{col_name}_{val}"] = (df[col_name] == val).astype(int)
    df.drop(columns=[col_name], inplace=True)
    return df

categorical_cols = ["season", "holiday", "workingday", "weather",
                    "month", "hour", "day", "year"]

for col in categorical_cols:
    df = one_hot_encode(df, col)

# print(df.head())
print(df.shape)
```

```
(10886, 74)
```

```python
# 80–20 split

X = df.drop(columns=["count"]).values.astype(float)
y = df["count"].values.reshape(-1, 1)
```

```python
def manual_split(X, y, test_ratio=0.2):
    n = len(X)
    idx = np.arange(n)
    np.random.shuffle(idx)

    split_point = int(n * (1 - test_ratio))
    train_idx = idx[:split_point]
    test_idx = idx[split_point:]

    return X[train_idx], X[test_idx], y[train_idx], y[test_idx]


X_train, X_test, y_train, y_test = manual_split(X, y, test_ratio=0.2)

print("Shapes:")
print("X_train:", X_train.shape)
print("X_test :", X_test.shape)
print("y_train:", y_train.shape)
print("y_test :", y_test.shape)
```

```
Shapes:
X_train: (8708, 73)
X_test : (2178, 73)
y_train: (8708, 1)
y_test : (2178, 1)
```

```python
# Why standardize
# If features are on very different scales, X transpose X becomes ill-
conditioned.
# Polynomials explode without scaling If humidity = 80:
# humidity⁴ = 40,960,000

def standardize(X_train, X_test):
    mean = X_train.mean(axis=0)
    std = X_train.std(axis=0) + 1e-8   # avoid division by zero

    X_train_std = (X_train - mean) / std
    X_test_std  = (X_test - mean) / std   # IMPORTANT: use TRAIN MEAN
+ STD

    return X_train_std, X_test_std, mean, std

X_train_std, X_test_std, mean_vec, std_vec = standardize(X_train,
X_test)

print("Standardized shapes:")
print(X_train_std.shape, X_test_std.shape)
```

```
Standardized shapes:
(8708, 73) (2178, 73)
```

```python
def add_bias(X):
    return np.hstack([np.ones((X.shape[0], 1)), X])

# Add bias column
X_train_b = add_bias(X_train_std)
X_test_b  = add_bias(X_test_std)

print("X_train with bias:", X_train_b.shape)
print("X_test with bias :", X_test_b.shape)
```

```
X_train with bias: (8708, 74)
X_test with bias : (2178, 74)
```

```python
# HELPER FUNCTIONS

# Solve for beta
def normal_equation(X, y):
    return np.linalg.pinv(X.T @ X) @ (X.T @ y)    #Moore–Penrose
pseudoinverse

# Prediction
def predict(X, beta):
    return X @ beta

# Mean Squared Error
def mse(y_true, y_pred):
    return np.mean((y_true - y_pred)**2)

# R² Score
def r2(y_true, y_pred):
    ssr = np.sum((y_true - y_pred)**2)
    sst = np.sum((y_true - np.mean(y_true))**2)
    return 1 - ssr/sst


# Train
beta_linear = normal_equation(X_train_b, y_train)

# Predict on test set
y_pred_linear = predict(X_test_b, beta_linear)

# Metrics
mse_linear = mse(y_test, y_pred_linear)
r2_linear = r2(y_test, y_pred_linear)

print("Baseline Linear Regression Results:")
print("MSE:", mse_linear)
print("R² :", r2_linear)
```

```
Baseline Linear Regression Results:
MSE: 9768.328489141288
R² : 0.7012313629107136
```

```python
def poly_no_interactions(X, degree):
    feats = [X]  # original features
    for p in range(2, degree + 1):
        feats.append(X ** p)   # element-wise exponent
    return np.hstack(feats)

def train_poly_model(degree, X_train_std, X_test_std):
    # Expand features
    Xtr_poly = poly_no_interactions(X_train_std, degree)
    Xte_poly = poly_no_interactions(X_test_std, degree)

    # Add bias
    Xtr_b = add_bias(Xtr_poly)
    Xte_b = add_bias(Xte_poly)

    # Train
    beta = normal_equation(Xtr_b, y_train)

    # Predict
    y_pred = predict(Xte_b, beta)

    # Metrics
    M = mse(y_test, y_pred)
    R = r2(y_test, y_pred)

    print(f"Polynomial degree {degree} (no interactions):")
    print("MSE:", M)
    print("R² :", R)
    print("-----------------------------------------------")

    return M, R

print("\n=== Polynomial Regression WITHOUT Interaction Terms ===\n")

results_poly = {}

for d in [2, 3, 4]:
    results_poly[d] = train_poly_model(d, X_train_std, X_test_std)
```

```
=== Polynomial Regression WITHOUT Interaction Terms ===

Polynomial degree 2 (no interactions):
MSE: 9696.038088486905
R² : 0.7034423967126744
-----------------------------------------------
Polynomial degree 3 (no interactions):
```

```
MSE: 9522.839305723564
R² : 0.7087397579069923
-----------------------------------------------
Polynomial degree 4 (no interactions):
MSE: 9489.167978435697
R² : 0.7097696103094752
-----------------------------------------------
```

```python
def quadratic_with_interactions(X):
    X = X.copy()
    N, d = X.shape

    # Linear + squared terms
    features = [X, X**2]

    # Interaction terms
    inter_terms = []
    for i in range(d):
        for j in range(i+1, d):
            inter_terms.append((X[:, i] * X[:, j]).reshape(-1, 1))

    # Combine all
    if inter_terms:
        inter_concat = np.hstack(inter_terms)
        return np.hstack([X, X**2, inter_concat])
    else:
        return np.hstack([X, X**2])

def train_quadratic_interaction_model(X_train_std, X_test_std):
    # Expand
    Xtr_q = quadratic_with_interactions(X_train_std)
    Xte_q = quadratic_with_interactions(X_test_std)

    # Add bias
    Xtr_b = add_bias(Xtr_q)
    Xte_b = add_bias(Xte_q)

    # Train using normal equation
    beta = normal_equation(Xtr_b, y_train)

    # Predict
    y_pred = predict(Xte_b, beta)

    # Metrics
    M = mse(y_test, y_pred)
    R = r2(y_test, y_pred)

    print("\nQuadratic Model WITH Interaction Terms:")
    print("MSE:", M)
    print("R² :", R)
```

```
    print("==============================================")

    return M, R

M_quad, R_quad = train_quadratic_interaction_model(X_train_std,
X_test_std)
```

```
Quadratic Model WITH Interaction Terms:
MSE: 2831.192171322513
R² : 0.9134067381841042
==============================================
```

```
results = {}

# Baseline Linear
results["Linear Regression"] = (mse_linear, r2_linear)

# Polynomial no-interactions
for d in [2, 3, 4]:
    M, R = results_poly[d]
    results[f"Poly degree {d} (no interactions)"] = (M, R)

# Quadratic with interactions
results["Quadratic (with interactions)"] = (M_quad, R_quad)

print(f"{'Model':40s} {'MSE':15s} {'R²'}")
print("-" * 70)

for model_name, (M, R) in results.items():
    print(f"{model_name:40s} {M:<15.4f} {R:.4f}")
```

```
Model                                    MSE             R²
----------------------------------------------------------------------
Linear Regression                        9768.3285       0.7012
Poly degree 2 (no interactions)          9696.0381       0.7034
Poly degree 3 (no interactions)          9522.8393       0.7087
Poly degree 4 (no interactions)          9489.1680       0.7098
Quadratic (with interactions)            2831.1922       0.9134
```